COMP521 Assignment 4 — Linux Memory Allocation and First-Touch Performance

Prediction

Before running the tests, I predicted that increasing the allocation size would have a larger impact on the first-time allocation compared to the allocation time itself. This is because in the C program, the touch loop accesses on byte for each memory page. Therefore when you have larger memory allocations, the program needs to access more pages when reading all the memory from the buffer specified in the function parameters. I expected allocation time to be the same because we are simply calling `malloc()` to give us a pointer to the newly allocated virtual memory.

Environment

For the experiment, I ran this on my Framework Laptop 13 running Arch Linux. At the time of running the experiment, my kernel version was `7.2.9-arch1-1`. The CPU on this laptop is an AMD Ryzen 5 7640U. The processor has 12 logical CPUS. 6 physical cores and 2 hardware threads per core. This laptop has 16GB (~14 GB usable) of RAM and there is no swap partition or swapfile configured (A swapfile was created for a different run where I allocated 20GB of memory). To get the system details, I ran the following commands with the following output

```
uname -a
nproc
free -h

Linux Briskycola-FWL 7.2.9-arch1-1 #1 SMP PREEMPT_DYNAMIC Sat, 03 Oct 2026 11:43:28 +0000 x8
12
               total        used        free      shared  buff/cache   available
Mem:            14Gi       3.5Gi       8.8Gi        89Mi       2.5Gi        11Gi
Swap:             0B          0B          0B
```

Experimental Design

The independent variable was the requested allocation size. The three allocation sizes were: 1. 16 MB 1. 64 MB 1. 128 MB 1. 20 GB (data not present in the submitted run)

The dependent variables were: 1. allocation time 1. first-touch time

The same C program, Bash script, and Linux environment, were used for every trial. Three trials were run for each allocation size, resulting in nine total data rows.

The program used `malloc()` to allocate memory to the heap. It returned a pointer to the virtual memory. The program then touched one byte (the buffer was defined as a `char` array) at the beginning of each page. The page size was obtained using `sysconf(_SC_PAGESIZE)`. The final byte of the allocation was also touched. The timing was measured using `clock_gettime()` with CLOCK_MONOTONIC. CLOCK_MONOTONIC represents the absolute elapsed time since a fixed point. On Linux, the starting point is actually when the system booted.

Results

The following table summarizes the three trials for each allocation size.

Allocation Size	Mean Allocation Time (seconds)	Minimum	Maximum	Range	Mean First-Touch Time (seconds)	Minimum	Maximum	Range
16 MB	0.0000137	0.000009	0.000017	0.000008	0.0016027	0.001515	0.001735	0.000220
64 MB	0.0000127	0.000009	0.000015	0.000006	0.0074557	0.007256	0.007560	0.000304
128 MB	0.0000100	0.000009	0.000011	0.000002	0.0134670	0.013423	0.013495	0.000072

The graph plots the mean allocation time and mean first-touch time for each allocation size. Because allocation time was much smaller than first-touch time, a logarithmic time axis was used (R supports logarithmic scale. `log = "y"` was used to display the y axis in logarithmic scale). so that both measurements could be displayed clearly.

The allocation-time means were approximately the same across all three conditions. The first-touch-time mean increased from approximately 0.0016 seconds for the 16 MB allocation to approximately 0.0075 seconds for the 64 MB allocation and approximately 0.0135 seconds for the 128 MB allocation.

Evidence-Based Analysis

Evidence

The allocation time results did not increase as the requested allocation size increased. The mean allocation times were approximately 0.0000137 seconds for 16 MB, 0.0000127 seconds for 64 MB, and 0.0000100 seconds for 128 MB. These differences were very small. The first-touch times increased substantially. The mean first-touch time increased from 0.0016027 seconds for 16 MB to 0.0074557 seconds for 64 MB and 0.0134670 seconds for 128 MB.

Meaning

Under the evidence compiled, increasing the allocation size had a much larger effect on first-time touch than on allocation time. The 128 MB allocation took approximately eight times longer to first-touch than the 16 MB allocation, while the measured allocation times remained nearly constant.

The first-touch time also increased in the expected direction between all three allocation sizes. The 64 MB allocation took longer than the 16 MB allocation, and the 128 MB allocation took longer than the 64 MB allocation.

Connection

This pattern relates to the difference between virtual allocation and physical memory residency. `malloc()` produced a usable virtual-memory region, but that alone does not mean that every byte will be immediately available in physical memory. The program's touch loop accessed one byte in each page. With a greater amount of memory allocated, there will be more pages as well as more pages to iterate through.

The larger allocations therefore required the program to visit more page-sized regions. The additional time may reflect page handling and page faults, along with allocator behavior, scheduling, timer overhead, and other system activity. Although this is not avaiable in the data collected for this run through, I had temporarily created a 10 GB swapfile in my system to conduct the experiment with extremely large allocations. I had allocated 20 GB of memory. The allocation time was identical to the previous results, however the first-touch time was nearly 15 seconds! This is due to the overhead that comes when swapping. The CPU does not have immediate access to the SSD or HDD and storage read and write speeds are significantly slower compared to RAM.

Limitations

This experiment had several limitations. Only three trials were performed for each allocation size. There could have been more trials with more variations to the allocation size. For example, there could have been a 512 MB or even a 1 GB trial. My main system also does not have swap space, so all allocations were done only in RAM. These results will most likely look different on a system with swapping (longer first-touch times due to disks having slower I/O). Some Linux distributions configure a swap partition (usually 2-5 GB) by default. I choose not to run swap storage since I never run out of memory and excessive swapping can create unnecessary writes to an SSD, reducing its lifespan.

Conclusion

Under the recorded conditions, increasing the requested allocation size had little effect on measured allocation time but substantially increased first-touch time. The larger allocations required the program to access more pages, which is consistent with the difference between obtaining a virtual-memory region and making pages physically resident through memory access. The results support the prediction that first-touch time would be more affected by allocation size than allocation time.